

Data model

course

course_id	integer	klucz główny (automatically incremented)
course_name	nvarchar(100)	
base_price	money	
planned_groups_amount	integer	default value 1
date_start	date	
date_end	date	
is_active	boolean	default value true

course enrollment

user_id	integer	reference to <u>users user</u>
group_id	integer	reference to <u>group</u>
enrollment_date	datetime	
total_cost	money	
discount_type	varchar(100)	default „bezwarunkowy”
discount_value	money	
is_completed	boolean	default value false
is_dropped	boolean	default value false

is_dropped=True jeśli user zrezygnował z opłat i uczestnictwa w zajęciach

group

group_id	integer	<u>klucz główny (automatically incremented)</u>
group_type	nvarchar(25)	default „zajęciowa”
course_id	integer	reference to <u>course</u>
max_group capacity	integer	

group timetable

group_id	integer	reference to <u>group</u>
room	nvarchar(10)	
datetime_start	datetime	
datetime_end	datetime	

users user

user_id	integer	klucz główny (automatically incremented)
email	nvarchar(255)	
first_name	nvarchar(200)	
last_name	nvarchar(200)	
is_active	boolean	
age	integer	

Zadanie 1 (5 punktów)

Proszę:

1. przygotować SQL script script_1.sql, we którym tworzą się bazę danych edu_courses, tabele oraz relację między nimi zgodnie z podanym modelem danych
2. przygotować SQL script script_2.sql, we którym zostaną
 - a. dodana kolumna: *phone_number* (varchar(25)) do tabeli users user
 - b. usunięta kolumna *age*
 - c. dodane sprawdzenie tego że *date_start* będzie zawsze mniejszą datą niż *date_end* w tabeli course
3. przygotować SQL script script_3.sql, we którym zostaną wstawiane przynajmniej 3 rekordy do każdej z tabel w modelu. Dane muszą być sensowne!

Proszę wysłać 3 pliki skryptów

Task 2 (5 punktów)

Proszę przygotować kod implementujący indeksy dla następujących wymagań (pamiętajcie o określeniu klucza indeksowania, unikalności):

1. Częste łączenie tabel users user i course enrollment.
2. Unikalność email.
3. Częste wyszukiwanie danych według daty rozpoczęcia i zakończenia kursu.
4. Złożony zgrupowany indeks w tabeli course enrollment.
5. Filtracja userów według imienia i nazwiska.

Proszę zapisać skrypt pod nazwą task 3.sql.

Zadanie 3 (10 punktów)

Proszę przygotować procedurę składowaną do zapisu użytkownika na wybrany kurs (pamiętajcie o zaprojektowaniu zakresu transakcyjnego):

1. Procedura przyjmuje: email oraz course_id.
2. Wykonuje walidację
 - a. czy podany użytkownik jest aktywny,
 - b. czy kurs jest aktywny,
 - c. czy jest chociaż jedno dostępne miejsce w grupach, przypisanych do danego kursuJeśli nie istnieje użytkownik o podanym email'u, trzeba go utworzyć nowego użytkownika
3. Jeśli parametry procedury spełnili warunki walidacji, to wtedy procedura oblicza koszt kursu zgodnie z kolejnymi zasadami:
 - a. użytkownik kupił pierwszy kurs w systemie – bezwarunkowy rabat w 100 zł,
 - b. użytkownik kupił drugi kurs w systemie – stały rabat w 5%
 - c. użytkownik kupił trzeci kurs (albo czwarty i td, **n**-ty kurs) w systemie – lojalnościowy rabat w **n**%, który trzeba dodać do stałego rabatu

i tworzy nowy rekord w tabeli course enrollment

Proszę zapisać skrypt pod nazwą procedure.sql. Również proszę dodać zrzut ekranu we którym demonstrują się tabele users user, course, course enrollment przed odpalaniem oraz po odpalaniu procedury. Bez zrzutów ekranu liczba punktów będzie mniejsza

Całkowite rozwiązanie powinno zawierać wszystkie trzy skrypty SQL w jednym pliku archiwum o nazwie zgodnej z nazwiskiem i imieniem (na przykład konetskaia_e.zip) i powinno zostać przesłane za pośrednictwem Moodle.